데이터베이스(SQL)

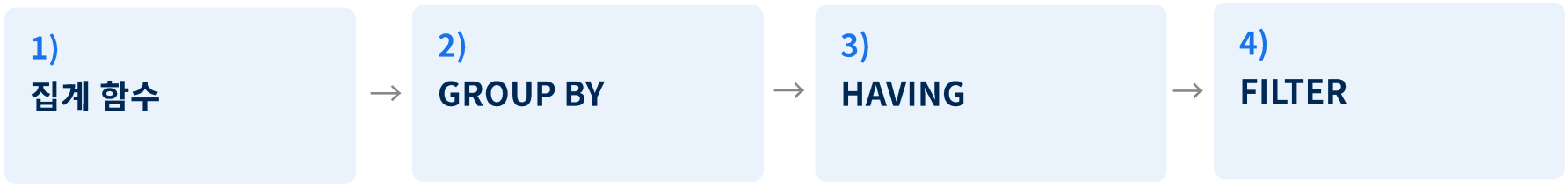
6장. SQL 집계 함수와 그룹 분석

학습 목표

1) 집계 함수(Aggregate Function)의 역할을 설명할 수 있다.
2) COUNT, SUM, AVG, MIN, MAX 함수를 사용할 수 있다.
3) COUNT(DISTINCT)로 고유값의 개수를 셀 수 있다.
4) 대상 행이 없을 때 집계 함수가 무엇을 반환하는지 설명할 수 있다.
5) NULLIF를 활용하여 0으로 나누기 오류를 방지할 수 있다.
6) GROUP BY를 이용하여 그룹별 분석을 수행할 수 있다.
7) HAVING을 이용하여 그룹 결과를 필터링할 수 있다.
8) WHERE와 HAVING의 차이를 설명할 수 있다.
9) FILTER로 조건부 집계를 작성할 수 있다.
10) 제조 데이터와 매출 데이터를 그룹 단위로 분석할 수 있다.

공식 문서: 집계 함수

전체 흐름



01 집계 함수

집계(Aggregation)란?

1) 총 매출은 얼마인가?

2) 고객은 몇 명인가?

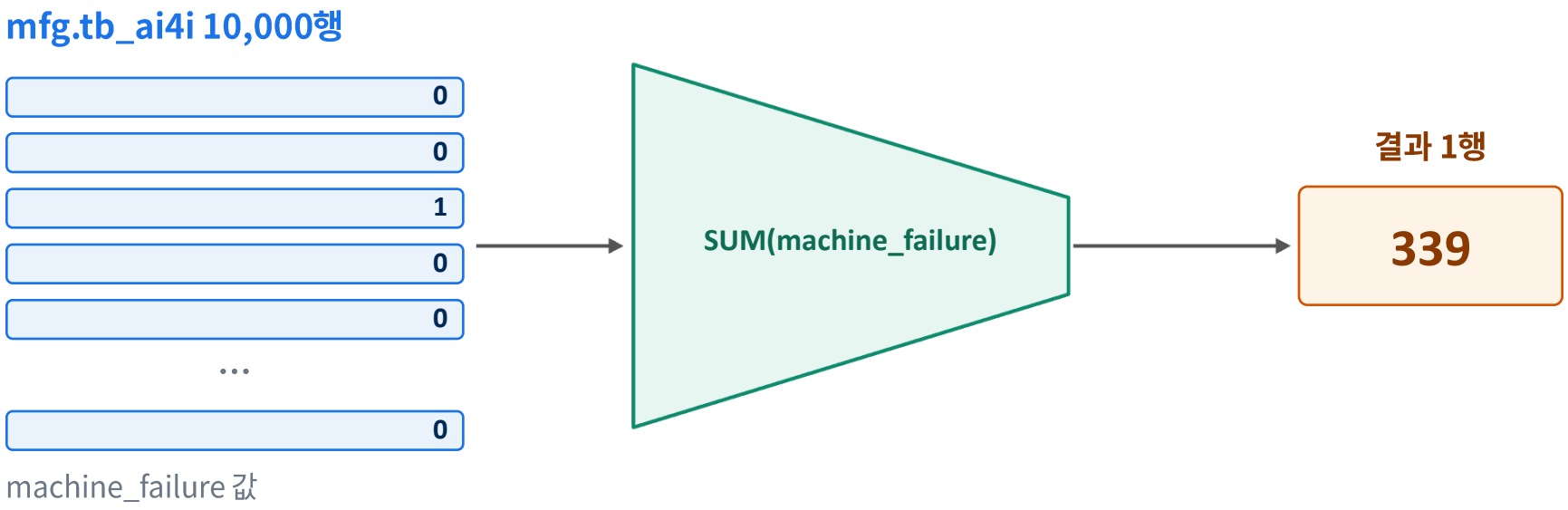
3) 제품 품질 유형별 고장 건수는 얼마인가?

4) 고장 관측 비율이 가장 높은 제품 품질 유형은 무엇인가?

이처럼 여러 행(Row)의 데이터를 하나의 결과로 요약하기 위해 사용하는 기능이 집계 함수(Aggregate Function)다.

공식 문서: 집계 함수 튜토리얼

집계: 여러 행을 하나로


mfg.tb_ai4i 10,000행
0
0
1
0
0
...
0
machine_failure 값
SUM(machine_failure)
결과 1행
339

집계 함수의 종류

집계 함수(Aggregate Function)는 여러 행의 값을 모아 하나의 결과를 계산한다.

예시 데이터 amount	10	20	NULL	30	총 4행

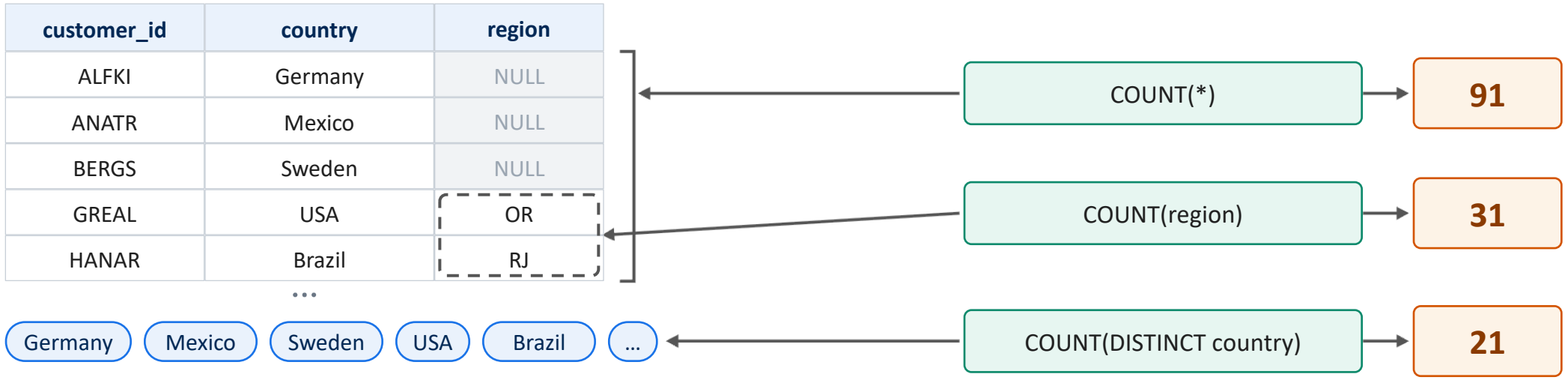
함수	역할	예시	결과
COUNT	행 수 계산	COUNT(*) / COUNT(amount)	**4 / 3**
SUM	합계	SUM(amount)	**60**
AVG	산술평균	AVG(amount)	**20**
MIN	최솟값	MIN(amount)	**10**
MAX	최댓값	MAX(amount)	**30**

COUNT(*)는 전체 행을 세고, COUNT(열)·SUM·AVG·MIN·MAX는 NULL 값을 제외한다.

공식 문서: PostgreSQL 18 — Aggregate Functions (9.21)

COUNT의 세 가지 형태

형태	세는 대상
COUNT(*)	전체 행
COUNT(컬럼)	그 컬럼이 NULL이 아닌 행
COUNT(DISTINCT 컬럼)	NULL을 제외한 고유값의 종류 수



COUNT - 개수

행(Row)의 개수를 계산한다.

COUNT(*) vs COUNT(컬럼명)

```
-- COUNT(*): NULL 포함 전체 행 수
SELECT COUNT(*) AS total_rows -- 전체 행 수
FROM customers;

-- COUNT(컬럼명): 해당 컬럼에 값이 있는 행만 계산 (NULL 제외)
SELECT COUNT(region) AS region_count -- region 값이 있는 행 수
FROM customers;
```

공식 문서: 집계 함수

COUNT(DISTINCT) - 고유값 개수

같은 값이 여러 번 나와도 한 번만 센다.

```
SELECT
    COUNT(country) AS total_count, -- 전체 건수: 91 (중복 포함)
    COUNT(DISTINCT country) AS country_count -- 국가 수: 21 (중복 제거)
FROM customers;
```

전체건수	국가수
91	21

고객은 91명이지만 국가는 21개다.
“종류가 몇개인가”를 물을 때 사용한다.

ROUND 자릿수 오류와 해결

```
-- torque_nm은 NUMERIC이라 AVG 결과도 NUMERIC → 그대로 된다
SELECT ROUND(AVG(torque_nm), 2) FROM mfg.tb_ai4i;

-- unit_price는 REAL이라 AVG 결과가 DOUBLE PRECISION → 오류
SELECT ROUND(AVG(unit_price), 2) FROM products;
-- ERROR: function round(double precision, integer) does not exist
```

한국어 Windows 환경

```
오류: round(double precision, integer) 이름의 함수가 없음
```

::NUMERIC으로 변환하면 해결된다.

```
SELECT ROUND(AVG(unit_price)::NUMERIC, 2) AS avg_unit_price -- 평균 단가
FROM products;
```

자릿수를 지정하는 ROUND는 NUMERIC 값에만 있다. 평균을 반올림할 때는 ::NUMERIC을 붙이는 습관을 들인다.

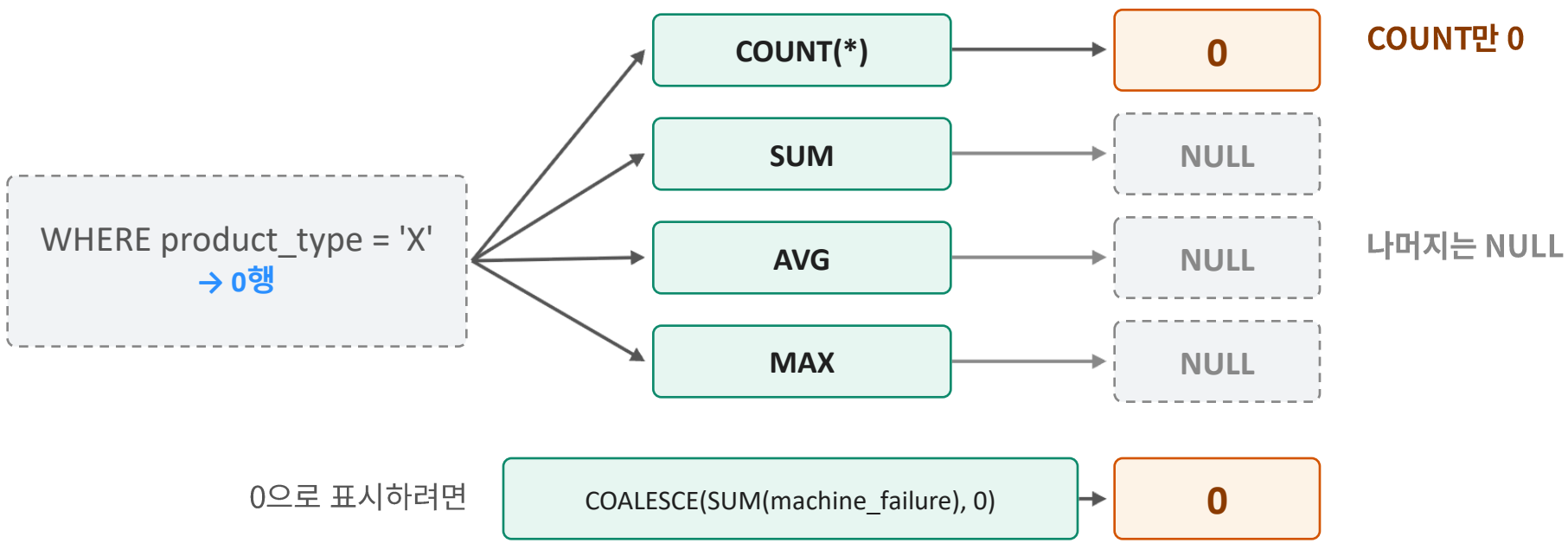
MIN과 MAX - 최소·최대

```
-- 제품 최저가와 최고가
SELECT
    MIN(unit_price) AS min_price, -- 최저가
    MAX(unit_price) AS max_price -- 최고가
FROM products;

-- 공구 마모 시간의 범위
SELECT
    MIN(tool_wear_min) AS min_wear, -- 최소 공구 마모 시간(min)
    MAX(tool_wear_min) AS max_wear -- 최대 공구 마모 시간(min)
FROM mfg.tb_ai4i;
```

MIN과 MAX는 숫자뿐 아니라 날짜와 문자에도 쓸 수 있다. 날짜의 MIN은 가장 이른 날짜, MAX는 가장 늦은 날짜다.

대상 행이 없을 때 집계 결과



대상 행이 없을 때의 결과

```
-- 존재하지 않는 제품 품질 유형 'X'로 조회
SELECT
        COUNT(*) AS row_count, -- 건수
        SUM(machine_failure) AS failure_sum, -- 고장 합계
        AVG(torque_nm) AS avg_torque, -- 평균 토크(Nm)
        MAX(tool_wear_min) AS max_wear -- 최대 공구 마모 시간(min)
FROM mfg.tb_ai4i
WHERE product_type = 'X';
```

건수	고장합계	평균토크	최대마모
0	(NULL)	(NULL)	(NULL)

공식 문서: 집계 함수

NULL 합계를 0으로 표시

```
SELECT
    COALESCE(SUM(machine_failure), 0) AS failure_sum -- 고장 합계
FROM mfg.tb_ai4i
WHERE product_type = 'X';
```

고장합계
0

02 GROUP BY와 처리 순서

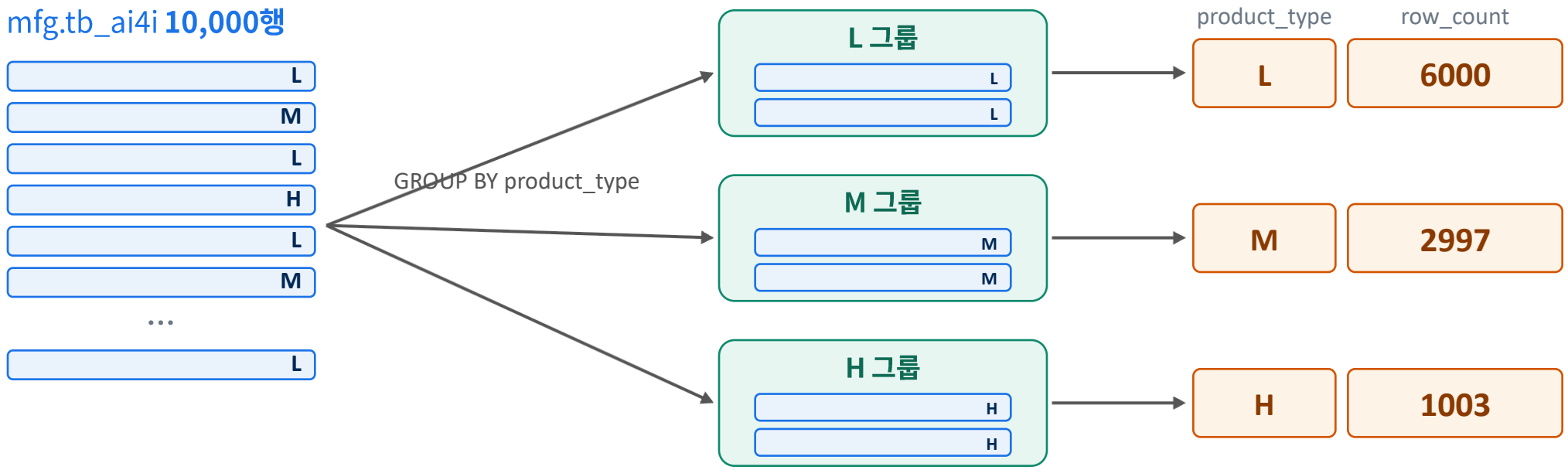
GROUP BY - 그룹별 집계

GROUP BY는 같은 값을 가진 행들을 하나의 그룹으로 묶는다.

```
-- 제품 품질 유형별 데이터 건수
SELECT
        product_type,
        COUNT(*) AS row_count -- 건수
FROM mfg.tb_ai4i
GROUP BY product_type;
```

공식 문서: GROUP BY와 HAVING

GROUP BY로 묶기



같은 값끼리 묶고, 묶음마다 한 행이 나온다

GROUP BY 활용 예

```
-- 국가별 고객 수
SELECT
	country,
	COUNT(*) AS customer_count -- 고객 수
FROM customers
GROUP BY country
ORDER BY customer_count DESC;
```

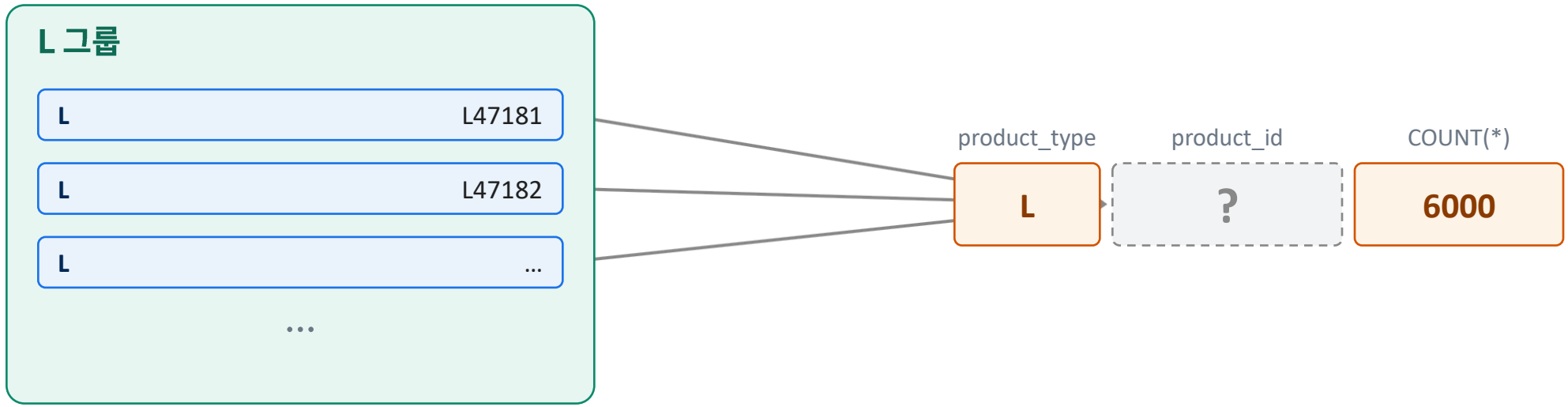
GROUP BY 오류

```
-- 잘못된 예: product_id는 GROUP BY에 없음
SELECT
product_type,
product_id, -- 오류 발생
COUNT(*)
FROM mfg.tb_ai4i
GROUP BY product_type;
-- ERROR: column "tb_ai4i.product_id" must appear in the GROUP BY clause
-- or be used in an aggregate function
-- (한국어 Windows: 오류: column "tb_ai4i.product_id" 는 반드시 GROUP BY 절내에 있어야 하던지
-- 또는 집계 함수 내에서 사용되어져야 한다)
```

product_type으로 그룹화하면 한 그룹 안에 여러 product_id가 존재한다.
데이터베이스는 그중 어떤 값을 출력해야 하는지 알 수 없으므로 오류가 발생한다.

SELECT 절의 일반 컬럼은 GROUP BY에 포함하고,
나머지는 집계 함수로 계산한다는 것이다

GROUP BY 오류의 이유



ERROR: column "tb_ai4i.product_id" must appear in the GROUP BY clause or be used in an aggregate function

한 그룹에 값이 여러 개인 컬럼은 하나를 고를 수 없다

GROUP BY 오류 예제 (2/2)

```
-- 올바른 예
SELECT
product_type,
COUNT(*) AS row_count -- 건수
FROM mfg.tb_ai4i
GROUP BY product_type;
```

처리 순서



SQL을 작성하는 순서와 처리되는 순서는 다르다. 적을 때는 SELECT가 맨 앞이지만, 처리는 FROM부터 시작한다.

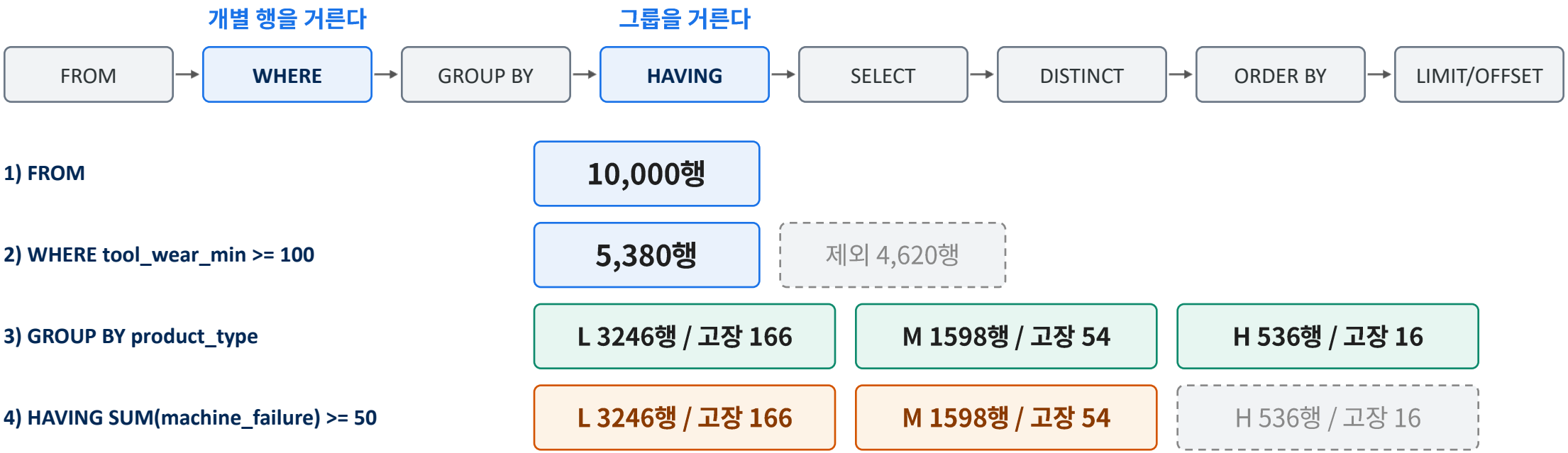
공식 문서: SELECT

처리 순서로 이해하는 규칙

1) WHERE는 그룹을 만들기 전에 개별 행을 거르므로 집계 함수를 직접 조건으로 사용할 수 없다.

2) HAVING은 만들어진 그룹을 대상으로 조건을 적용할 수 있다.

WHERE와 HAVING의 위치



03 HAVING

HAVING - 그룹 결과 필터링

GROUP BY 결과에 조건을 적용하기 위해 사용한다.

```
-- 데이터가 3,000건 이상인 제품 품질 유형
SELECT
        product_type,
        COUNT(*) AS row_count -- 건수
FROM mfg.tb_ai4i
GROUP BY product_type
HAVING COUNT(*) >= 3000;
```

```
-- 할인 전 금액 합계가 5,000 이상인 주문만 조회
SELECT
        order_id,
        SUM(unit_price * quantity) AS order_amount -- 주문별 할인 전 금액
FROM order_details
GROUP BY order_id
HAVING SUM(unit_price * quantity) >= 5000
ORDER BY order_amount DESC;
```

공식 문서: GROUP BY와 HAVING

WHERE와 HAVING 비교

구분	WHERE	HAVING
적용 시점	GROUP BY 전	GROUP BY 후
대상	개별 행	그룹 결과
집계 함수 사용	불가	가능
예시	마모 시간 100 이상인 행	그룹의 총건수 1000 이상

04 FILTER

FILTER가 필요한 이유

```
-- CASE WHEN 방식 (기존 방법)
SELECT
COUNT(CASE WHEN product_type = 'H' THEN 1 END) AS h_count, -- H 유형 건수
COUNT(CASE WHEN product_type = 'M' THEN 1 END) AS m_count, -- M 유형 건수
COUNT(CASE WHEN product_type = 'L' THEN 1 END) AS l_count -- L 유형 건수
FROM mfg.tb_ai4i;
```

공식 문서: 집계 표현식 FILTER 절

FILTER 문법

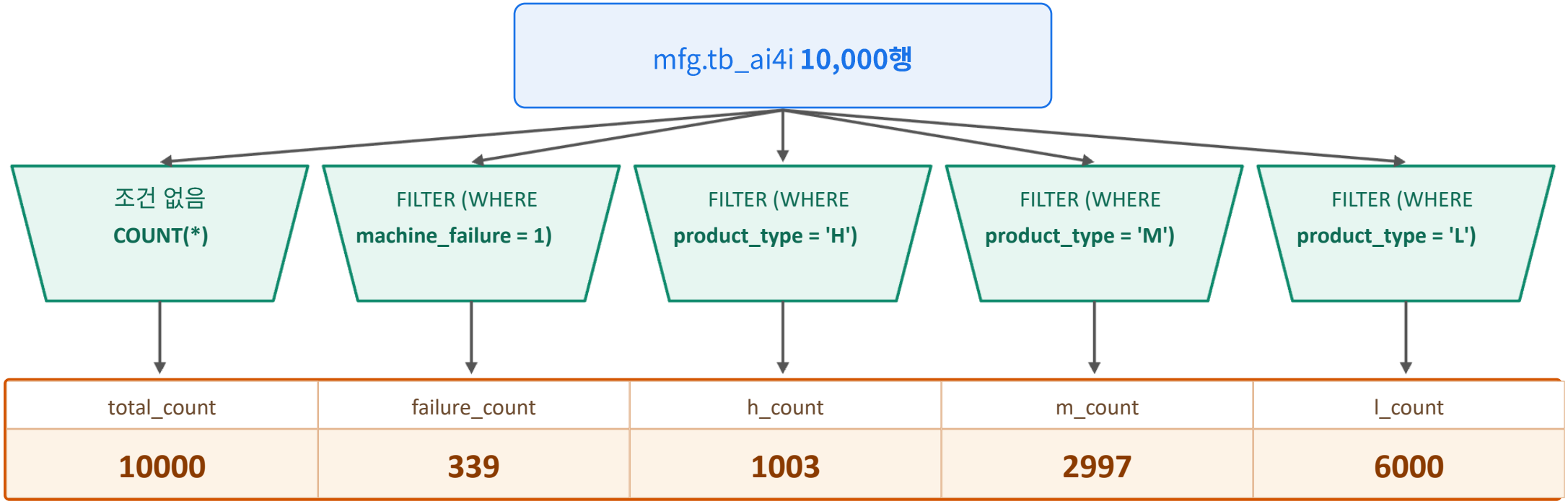
집계 함수에서 지정한 조건을 만족하는 행만 집계하도록 하는 SQL 조건부 집계 구문이다

집계함수(컬럼)	FILTER	(WHERE 조건)
계산할 집계	입력 행을 제한하는 키워드	집계에 넣을 행의 조건

FILTER 예제 - 전체 요약을 한 행으로

```
-- 전체 요약을 한 행으로
SELECT
	COUNT(*) AS total_count, -- 전체 건수
	COUNT(*) FILTER (WHERE machine_failure = 1) AS failure_count, -- 고장 건수
	COUNT(*) FILTER (WHERE product_type = 'H') AS h_count, -- H 유형 건수
	COUNT(*) FILTER (WHERE product_type = 'M') AS m_count, -- M 유형 건수
	COUNT(*) FILTER (WHERE product_type = 'L') AS l_count, -- L 유형 건수
	ROUND(
		COUNT(*) FILTER (WHERE machine_failure = 1)::NUMERIC
		/ COUNT(*) * 100, 2
	) AS failure_rate_pct -- 전체 고장률(%)
FROM mfg.tb_ai4i;
```

FILTER의 조건부 집계



total_count	failure_count	h_count	m_count	l_count
10000	339	1003	2997	6000

FILTER는 그 집계 함수에 들어가는 행만 제한하고, 다른 열의 집계에는 영향을 주지 않는다.

CASE WHEN과 FILTER 비교

항목	CASE WHEN	FILTER
표현 방식	CASE 결과를 집계 함수에 전달	집계 함수 입력 행을 조건으로 제한
가독성	조건이 많아지면 길어질 수 있음	집계 조건이 바로 보여 읽기 쉬움

핵심 정리 (1/2)

1) 집계 함수는 여러 행을 하나의 결과로 요약한다.
2) COUNT(*)는 NULL을 포함한 전체 행 수를 계산하고, COUNT(컬럼)은 NULL을 제외한다. COUNT(DISTINCT 컬럼)은 고유값의 종류 수를 센다.
3) SUM은 합계, AVG는 평균, MIN은 최소값, MAX는 최대값을 계산한다.
4) 0/1 컬럼은 SUM이 건수, AVG가 비율이 된다.
5) AVG는 NULL 값을 제외하고 평균을 계산한다.
6) 대상 행이 하나도 없으면 COUNT만 0을 반환하고 SUM, AVG, MIN, MAX는 NULL을 반환한다. 0으로 표시하려면 COALESCE로 감싼다.
7) ROUND()에서 소수 자릿수를 지정할 때, 계산 결과가 DOUBLE PRECISION이면 NUMERIC으로 형변환해야 한다.
8) NULLIF(값, 0)은 값이 0이면 NULL을 반환하여 0으로 나누기 오류를 방지한다.

핵심 정리 (2/2)

9) ::NUMERIC은 정수 나눗셈에서 소수점 결과를 얻기 위한 형변환이다.

10) 비율 계산의 기본 패턴은 분자::NUMERIC / NULLIF(분모, 0) * 100이다.

11) GROUP BY는 같은 값을 가진 행을 그룹으로 묶는다. SELECT의 일반 컬럼은 GROUP BY에 포함한다.

12) WHERE는 그룹화 전 개별 행을 거르고, HAVING은 그룹화된 결과를 거른다.

13) SELECT의 별칭은 GROUP BY와 ORDER BY에서 쓸 수 있고 WHERE와 HAVING에서는 쓸 수 없다.

14) 별칭이 원본 컬럼명과 겹치면 원본 컬럼으로 해석된다. 겹치지 않는 이름을 쓴다.

15) FILTER (WHERE)는 해당 집계 함수에 입력되는 행만 조건으로 제한하여 조건부 집계를 간결하게 표현한다.